

Scanning range

Field manual: a different network every time, scored on what you find

Overview

This is a practice range rather than a walkthrough. Every spawn draws a fresh network from a seed: a different graph of routers and segments, different subnets inside a different /16, between six and nine machines scattered across those segments, a service drawn for each from a palette of seventeen, a different set of weaknesses in those services, a different organisation whose network it is, and a different opening move. Nothing is fixed except the rules the draw obeys, and this document is those rules plus the syntax of the tools you will need. There are no questions in it, because a question needs a fixed answer and this exercise has none.

The range is scored in two halves. The survey is the larger one: which addresses hold a machine, what kind of machine each is, which ports on them have a process listening, and what software is behind each. The second half is shorter and it is what the survey is for. One of the services you find names an account, another holds its password or the passphrase on a key, and between two and four such steps in is a flag. Which services carry which of those is drawn per spawn, and above the easy difficulty more than one route exists to each.

Definition: Reconnaissance

The stage of an attack that runs before anything is exploited: establishing which addresses in a target prefix carry a host that replies to a given probe, which transport-layer ports on those hosts have a process listening, and which software is behind each listener. A host that replies to nothing you send has not been shown to be absent, only to be unmeasured by the probes you used. Reconnaissance produces a target list, not access.

What varies between spawns is the whole of the network: the number of routers, how many segments hang off each, which /24 each segment uses, which addresses inside it hold a machine, what kind of machine it is, which service it runs and which implementation of that service, whether the service is on the port its registry entry names, whether the machine replies to an echo request, whether its shut ports send a reset or nothing at all, which findings are there to be made, how far in the chain runs, and what you are told at the start. One further word is drawn that changes how the whole range behaves rather than what is on it, and Status prints it.

Four things do not vary. Every segment's router interface is the .1 of that segment. Exactly one machine in the range runs no service at all. The chain always begins at a login prompt on a telnet service. And the scoring weights below are the same every time.

The range is Layer 3 and 4 only. Every probe it rewards is an ICMP message, a TCP segment or a UDP datagram, and the services it draws reply on TCP or on UDP. Six of the seventeen are UDP, so a survey that scans TCP alone misses part of every range. It does not draw anything reachable only over QUIC, over IPv6, or over a link-layer protocol other than Ethernet, so a technique aimed at any of those finds nothing here and that is a property of the range rather than of scanning.

Running this range in the TUI

You drive the range from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab    # first time
cd ~/MiniLab && git pull                                       # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

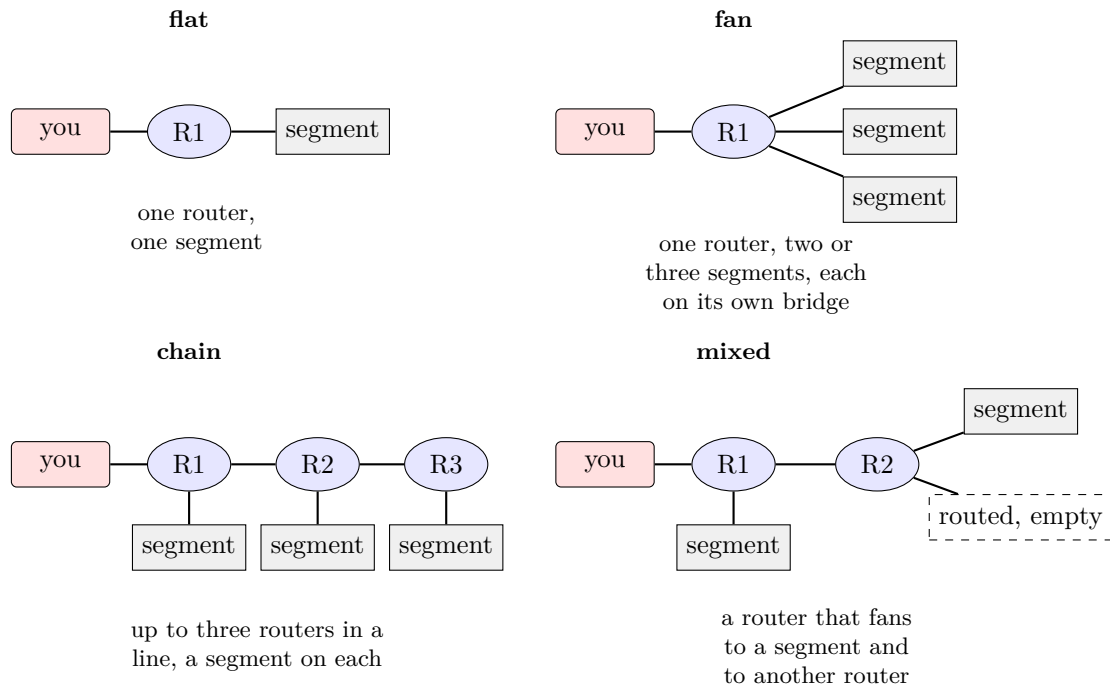
The root menu lists one category per row. Select **Practice ranges**, then **Scanning range (randomised)**, to reach these actions:

- **Spawn** asks for a difficulty and then for a seed. Leave the seed blank and you get a network nobody has seen before; type the number a previous run printed and you get that same network back. Spawn then creates the containers and the wiring. Wait for the panel to turn green before you go on. The first spawn also builds the image, which takes a few minutes; later spawns take seconds.
- **Status** prints the seed, the address and gateway you have been given, the one word that says how this range behaves, and whether every machine the range spawned is still healthy. It prints nothing about what is out there, because that is the exercise. It takes over the whole screen: the arrow keys scroll, and Enter on **Done** closes it.
- **Shell command for a node** lists the nodes it accepts and the `docker exec -it ...` line that opens a shell on each. Until you have run Score that list is **attacker** alone, which is where every step below runs: naming the rest would name one container per machine, and how many machines are out there is part of what you are scored on. It holds the screen until Enter on **Done**, as Status does.
- **Score** grades what you have recorded and prints your marks category by category. You can run it as often as you like, and running it does not end the attempt.
- **Reveal** prints the whole answer beside what you submitted. It stays locked until you have run Score once on this network.
- **Regenerate** throws this network away and draws a fresh one at the same difficulty.
- **Teardown** removes the range.
- **Open handout** opens this document.

Spawn a range now, then work through it in your own terminal. **Back** (or Esc) returns to the list.

The shapes a range can take

Before it draws a single address the seed draws the graph, which is one of four. You are not told which one you were handed. Working it out is part of the exercise at every tier, and at **hard** it is where the exercise starts.



Two properties of every shape are worth holding on to. A segment is a broadcast domain of its own: hosts on one segment cannot be reached by an ARP request sent from another, and only the router between you and them carries your packets across. And the dashed box in the fourth drawing is a subnet that a router has an interface on and no host sits in. A sweep of one finds nothing, and it is slow: the router answers each probe with an ICMP host unreachable, and Linux emits those no more than one a second by default (`net.ipv4.icmp_ratelimit`, in milliseconds), so 254 addresses take around four minutes to come back empty. Four probes tell you the same thing in four seconds.

What you are given, by difficulty

Difficulty	Shapes	Segments	Hosts	Layers in
easy	flat, fan	1 to 2	6 to 7	2
normal	fan, chain	2 to 3	6 to 8	3
hard	chain, mixed	3	7 to 9	3, or 4 on mixed

The size of a range is not what makes it harder. What changes with difficulty is how much you are told at the start, and Status prints it. The opening is drawn per spawn from the list its difficulty allows, so two ranges at one difficulty differ in the first move as well as in the map.

Opening	Drawn at	What you are handed
segs	easy	every live segment's /24, confirmed. Scores nothing
candidates	easy, normal	a list of /24s, two of which hold nothing
count	easy, hard	the /16, and how many live segments are in it
prefix	normal	the /16
dns	normal	the address of the host authoritative for the zone
nothing	hard	your own address and gateway, and not one thing more

Only **segs** costs you marks by being given: it names segments that are confirmed live, so finding them is not work. Under **candidates** you are handed blocks of which two do not exist, so establishing which are real scores in full. Under **count** the number tells you when to stop looking, not where to look.

Where you are given nothing, the second octet of your own address is the handle: every subnet a range draws sits inside the /16 that your own interface is in, so `ip addr show` gives you the space to search and `ip route show` gives you the one router you already know about.

```
# 116_L4_RANGE_attacker: the two commands that start a hard range
ip -brief addr show      # your own address and prefix; the /16 is the first two
                          octets
ip route show            # your default gateway: R1, and the only router you are
                          given
```

How this one behaves

One more word is drawn per spawn, and Status names it. It changes how the whole range answers rather than what is on it, so two ranges with the same shape, the same machine count and the same services are still two different afternoons. Knowing which one you drew tells you which of your habits is about to be wrong; it tells you nothing about where anything is.

Word	What it changes
vanilla	nothing
quiet	most machines drop echo requests, so a ping sweep finds almost nothing and a TCP probe finds them
noisy	three machines hold six to eight extra listeners that accept a connection and then send nothing
relocated	every service that can be moved is on one of its alternate ports
chatty	an SNMP agent answers on most machines, so their interface and address tables are a second route to the map
locked down	shut ports are dropped rather than refused nearly everywhere, so a scan reports filtered where it usually reports closed
legacy	telnet, TFTP and FTP heavy, and no SSH anywhere on the range
segmented	an extra routed-and-empty subnet, and one of the empty ones answers an administrative rejection rather than a host unreachable

How the range is scored

Recording a finding

Findings are recorded with `report`, a command on the attacker. It appends a line to `/root/findings.txt` and checks only the shape of what you typed. It does not tell you whether a finding is right, in its output or in its exit status, so a line it accepts is not a line that scores.

```
# 116_L4_RANGE_attacker
report net    116.19.0.0/24 hops 2      # a segment, and its distance from you in
routers
report router 116.7.0.1                # an address you believe is a router
interface
report host   116.7.0.23                # an address you believe holds a host
report port   116.7.0.23 tcp 80 http lighttpd 1.4.85
report intel  116.7.0.87 anon-ftp       # one of the tokens listed further down
report class  116.7.0.23 server         # server, workstation, netdev or appliance
report flag   MINILABS{foothold-1a2b3c4d} # a token off a machine you got into
report list
report del 4
report clear
# everything you have recorded, numbered
# remove line 4
# start again
```

On a `port` line the service name and the version are both optional and both scored separately, so a port you found but could not identify is still worth recording. The version field is free text: the scorer takes the first run of digits and dots out of it, so `lighttpd 1.4.85`, `1.4.85` and `lighttpd/1.4.85` are the same answer, and `1.4` is accepted against a truth of `1.4.85`. Service names are matched without regard to case and each port accepts several: `http` and `lighttpd` both count on a web port, `domain`, `dns`, `bind` and `named` all count on a DNS one.

The weights

Category	Found	Wrongly claimed	Notes
subnet	+4	-3	a live segment; a routed-and-empty one is not
hops	+2	none	scores only on a segment you already found
router	+3	-3	any router interface but your own gateway
host	+2	-2	an address that holds a machine
port	+3	-1	open; a filtered or closed one is not
service	+2	none	only on a port that is genuinely open
version	+3	none	only where the service announces one
intel	+5	-3	the tokens below
class	+2	-2	what kind of machine an address holds
foothold	+6	none	the first machine you get a shell on
pivot	+10	none	a shell on a machine past a router
vault	+14	none	a shell on the last machine on the chain
flag	+20	none	the token behind the sudo rule on it

The total is scaled to 100 against what the range actually held, so a score is the fraction of that network you found and is comparable between two seeds. The floor is 0: a submission of every address in a segment cannot score below an empty one, and the penalties are what make it score worse. Around half to two thirds of the marks sit in hosts, ports, services and versions, and the last four rows are worth between a fifth and a third depending on how deep the range runs. Scanning carefully is most of the exercise; the chain is what separates two learners who both scanned it well.

Two addresses are outside the scoring entirely, neither credited nor penalised: your own address, and your own default gateway. Both are on your own interface and in your own routing table, so neither is something a scan found. Every other router interface counts, including the `.1` of a segment you have just discovered and the two ends of each router-to-router link, which only a traceroute will show you. A `class` line on a router interface is ignored in both directions for the same reason: a router is a network device, and the range does not classify the addresses it

did not put a machine on.

Every category that can be claimed wrongly costs marks, and that is what makes reporting everything worse than reporting nothing. A **class** on an address that holds no machine costs two, so guessing **server** down a whole /24 loses more than it collects.

The intel tokens

Each of these needs you to talk to a service rather than probe it, and each is worth five points on the machine that carries it and costs three on a machine that does not. No range holds all sixteen; which ones it holds follows from which services it drew.

Token	What it means
anon-ftp	the FTP service accepts a login as anonymous
leaked-account	a file on that share names an account used elsewhere in the range
leaked-passphrase	a file names the passphrase on a private key
leaked-key	a private key is sitting in a home directory you reached
hidden-path	the web service serves a path its index page does not link to
default-cred	the credential the device in the telnet banner ships with still works
weak-telnet-pass	that shared account's password is in the wordlist on your machine
zone-transfer	the DNS service hands its whole zone to anyone who asks
snmp-public	the SNMP agent answers the community string it shipped with
snmp-map	its address table names a block that is not one of the segments
smtp-vrfy	the mail service confirms which of a list of names is an account
tftp-file	a file is readable over TFTP once you know its name
redis-open	the key-value store answers with no authentication
rsync-module	the file-sync daemon lists a module to an anonymous client
mqtt-open	the message broker accepts a subscription with no credential
open-proxy	the proxy makes a connection on behalf of anyone who asks

Four things on the range that are not findings

The penalties above are not decoration. Every range plants some of the following, and each is something that looks like a finding until you probe it.

- Two names in the transferred zone point at addresses that hold nothing. A zone is a records file somebody wrote, not a list of what is running, so reporting an address because a record names it costs two points each time.
- An SNMP address table can name a block that is routed nowhere. The interface holding it is administratively down. Reporting the block as a segment costs three.
- A **robots.txt** can name a path the web server answers 404 for. The finding is what a **GET** returns, not what a text file asserts.
- A token shaped exactly like a real one can be sitting on the anonymous share, in a file whose first line says the environment it came from was torn down. It matches nothing and scores nothing, and reading two lines is what tells you so.

One web service on some ranges announces no version in its **Server** header, and one router on some ranges sends no ICMP time-exceeded, so a traceroute through it shows a gap where a hop should be. Neither is a fault. Count the gap as a hop: the numbering in the left column is still right.

The chain, and the flag at the end of it

The intel tokens above are things you learn about a service. The chain is what one of those services is good for. Every range carries one, it runs between two and four layers deep, and Status prints which. At the end of it is a flag.

Every door on the chain opens because of a configuration somebody left wrong: a device still holding the credential it shipped with, an account whose password is on a public list, a private key sitting in a home directory, that key's passphrase written down on a share that asks for no login, and a `sudo` rule granting one account the right to read one file as `root`. None of it is an exploit against a software defect, and none of it needs one. Each is fixed by editing a file.

You are paid for depth reached, not only for finishing. A learner who takes the first shell and gets no further keeps those six points.

Milestone	Worth	What it means
foothold	+6	a shell on the telnet host
pivot	+10	a shell on a machine the first one's key opens
vault	+14	a shell on the last machine on the chain
flag	+20	the token the <code>sudo</code> rule on that machine reads out

Each milestone is a file on the machine it belongs to, holding one token that looks like `MINILABS{foothold-1a2b3c4d}`. Submit it with `report flag`. You do not say which milestone you think it is: the token identifies itself. A token that matches nothing scores nothing and costs nothing, so a typo is free, and `report` takes twenty of them before it stops accepting more.

Definition: Public-key authentication, and a passphrase

An SSH server can be configured to accept no password at all and to admit only a client holding the private half of a key pair whose public half is listed in that account's `~/.ssh/authorized_keys`. The private key is a file. When it is generated with a passphrase, the file on disk is encrypted with it, so possession of the file is not possession of the key: `ssh` asks for the passphrase before it can use it, and `ssh-add` loads it into an agent once so the following connections do not ask again.

Finding the key and finding its passphrase are two different pieces of work in two different places, and neither one alone opens anything. That is deliberate. A learner who cracked a login and never enumerated the shares has a key file they cannot decrypt.

Definition: ICMP destination unreachable, code 13

A router that forwards a packet answers with an ICMP type 3 message when it cannot deliver it, and the code says why. Code 0 is network unreachable and code 1 is host unreachable, which are both a router saying the destination is not there. Code 13 is **communication administratively prohibited**: the router can reach the destination and a filter on it decided not to. `nmap` reports a port filtered this way as `admin-prohibited`, and `traceroute` prints `!X` at that hop.

The difference between code 0 and code 13 is the difference between empty space and a locked door, and one of the segments on a range at three layers or deeper answers with the second. That subnet is outside the scoring in both directions, like your own address and your own gateway, so probing it and reporting it costs you nothing either way.

What kind of machine an address holds

Every machine on the range is one of four kinds, and **report class** is worth two points on each and costs two for a wrong one. There is no scanner flag for this. **nmap -O** fingerprints the IP stack and every machine here runs Linux, so it answers Linux fifteen times and settles nothing. The class is an inference from four weak signals, none of which is conclusive on its own, and that is what fingerprinting actually is.

Definition: Initial TTL

The value a host writes into the IP time-to-live field of a packet it originates. Each router that forwards the packet decrements it by one, so the value you receive is the sender's initial value minus the number of routers between you. Operating systems and embedded firmware differ in what they start at, and the common values are 64, 128 and 255. A reply arriving with a TTL of 126 from a machine two routers away started at 128.

Definition: OUI

The first three octets of a MAC address, assigned to one manufacturer by the IEEE and published, so a MAC identifies who built the interface. It is only readable where ARP reaches, which is the segment you are standing on: a reply from behind a router carries the router's hardware address, not the sender's.

Class	TTL / OUI	What it runs
server	64 / 00:14:22	web, FTP, SSH, DNS, mail, key-value store, file sync, broker, proxy
workstation	128 / 3c:97:0e	very little, and one of them runs nothing at all
netdev	255 / 00:1b:0d	telnet, SNMP, TFTP, a log collector
appliance	64 / 00:c0:b7	telnet, a small web page, SNMP, a print port, a monitoring agent

The TTL separates three of the four and leaves **server** and **appliance** sharing 64, which is the point: no single signal decides it. The other three are the machine's own name, which it gives you in a telnet banner and in a transferred zone, its SNMP **sysDescr** where it runs an agent, and the service mix itself. A machine answering on port 9100 and nothing else is not a file server whatever its TTL says.

```
# 116_L4_RANGE_attacker: the two signals you read rather than infer
ping -c 1 116.7.0.23          # the reply line ends with ttl=; add the hop
                             count back on
ip neigh show 116.0.0.210     # the MAC, for a machine on your own segment only
snmpget -v2c -c public 116.7.0.23 sysDescr.0 # what the machine says it is
```

Par time, and Reveal

Score prints how long the range has been up beside a par time for the difficulty: twenty minutes at **easy**, thirty at **normal**, forty-five at **hard**. Par does not change your score. It is there so you can see whether you are working at the pace the range was built for.

Score can be run as often as you like and running it does not end the attempt. The first run unlocks Reveal, which prints the whole network beside what you submitted and names everything you missed. Regenerate then draws a fresh range at the same difficulty and locks

Reveal again.

Tool reference

Everything below is on the attacker already. This section is reference rather than a walkthrough: come back to it for the flag you have forgotten.

Every address printed in this document is an example. None of them is on your range, and a range that happens to draw one is a coincidence rather than a hint. Substitute the addresses your own scans return.

Definition: Standard output and standard error

Every command has two output streams. Standard output carries what the command was asked to produce; standard error carries its diagnostics. `> file` redirects standard output alone, and `2>/dev/null` discards standard error alone, so the two can be separated. Which stream a particular tool writes progress to is that tool's own choice and is noted below where it matters.

zmap: sweep a whole prefix

Definition: zmap

A single-packet scanner. It sends one probe per address across a whole prefix without waiting for replies, and prints on standard output the address of every host that replied. Its default probe module is `tcp_synscan`, which sends a TCP SYN to a port you name with `-p` and prints only the addresses that came back with a SYN-ACK: a reset counts as a failed response and is not printed, even though a reset also proves a host is running. `-M icmp_echoscan` sends an ICMP echo request instead. Progress lines go to standard error, which is what makes the redirection below leave a file of nothing but addresses.

```
# 116_L4_RANGE_attacker
zmap -M icmp_echoscan 116.7.0.0/24 2>/dev/null > /root/scan/icmp.txt
zmap -p 80 116.7.0.0/24 2>/dev/null > /root/scan/tcp80.txt    # default module: a
                    TCP SYN
zmap -M icmp_echoscan 116.0.0.0/16 -r 50000 2>/dev/null      # a /16 at 50000
                    packets/second
```

`-r` sets the transmit rate in packets per second, and `-B` sets it in bits per second; setting both is refused. Neither flag prints a default and the unset case is easy to read as unlimited, which it is not: ZMap 3.0 sends at 10,000 packets per second when it is given neither, so a /16 left alone takes around seven seconds and the line above takes under two. A prefix handed to a scanner is probed in full: a /24 is 256 probes, not the 254 addresses that can be assigned to an interface, because the scanner does not skip the network and broadcast addresses. zmap prints the number it sent on its progress line, so the two numbers you see will differ and both are right about different things.

nmap: identify what is behind a port

Definition: nmap

A scanner that probes a host over several protocols and reports a state per port. `-sn` does host discovery only and no port scan; run by a privileged user against a target off the local segment its default probe set is an ICMP echo request, a TCP SYN to port 443, a TCP ACK to port 80, and an ICMP timestamp request, and any one reply is enough to report the host as up. On the local segment it uses an ARP request instead, which `--disable-arp-ping` switches off. `-p` names ports, `-sV` identifies the software behind them, and `-n` skips reverse DNS lookups, which are PTR queries for the name belonging to an address and are a different record type from the A-record lookups that resolve a name.

```
# 116_L4_RANGE_attacker
nmap -sn -n 116.7.0.0/24          # host discovery, default probe set
nmap -sn -PE --disable-arp-ping -n 116.7.0.0/24 # echo requests only, and nothing
else
nmap -p- -n 116.7.0.23           # every one of the 65535 TCP ports
nmap -sV -n -p 21,80,2100 116.7.0.23 # identify the software behind
three ports
nmap -sU -n -p 53 116.7.0.88      # UDP; slower, and needed for the
DNS host
```

Definition: Port states: open, closed, filtered

open means a process accepted the probe. **closed** means the host replied that nothing is listening: for TCP that is a reset segment. **filtered** means nothing came back at all, so the scanner cannot tell a host that is discarding the probe from one that never received it. The difference between **closed** and **filtered** is a property of the host you are scanning and not of your scan, and this range draws it per host: some of them reply to a shut port with a reset and some drop it.

RFC 9293 requires a TCP endpoint with no listener on the target port to reply with a reset rather than stay silent. That is a requirement on the endpoint and not a prediction about your screen: it holds only where the segment reaches a host that implements the requirement and nothing in front of it is configured to drop it. Where it does hold, a TCP probe may establish that a host is running even when the host replies to no ICMP at all, which is exactly how the hosts in this range that a ping sweep misses are found.

Scanning UDP, and what open|filtered means

Six of this range's seventeen services are UDP: SNMP on 161, TFTP on 69, NTP on 123, a log collector on 514, and DNS on 53, which answers on both transports. A survey that scans TCP alone misses them and reports the machines carrying them as having nothing open.

Definition: open|filtered

UDP has no handshake, so a datagram sent to a port where a process is listening and choosing not to reply produces the same silence as one a firewall dropped. **nmap** reports the port as **open|filtered** because it genuinely cannot tell the two apart from that probe alone. It reports **closed** only when the host answers with an ICMP port unreachable, and **open** only when the service itself replies.

What separates the two is the ports around it. A machine that answers **closed** on its other UDP ports is one that sends ICMP port unreachable, so a port on it that stays silent has something bound to it. A machine that reports **open|filtered** on every port tells you nothing, and on a **locked down** range that is most of them.

Two things make a UDP sweep slow, and neither is the scanner. Linux rate-limits ICMP destination unreachable messages, at one a second by default (`net.ipv4.icmp_ratelimit`, in milliseconds), so the **closed** answers arrive one per second however fast you send. And nmap retries a silent port before it gives up. Name the ports you care about rather than sweeping all 65535, and give the scan the ports the services above actually use.

```
# 116_L4_RANGE_attacker
nmap -sU -n -p 53,69,123,161,514 116.7.0.23      # the five UDP ports this range
draws
nmap -sU -sV -n -p 161 116.7.0.23              # and identify what answered
```

masscan: a second opinion, quickly

Definition: masscan

Another single-packet scanner. It sends TCP SYNs to the ports you name across the prefix you name and prints the ones that replied. Its own default transmit rate is 100 packets per second, so `--rate` on the line below raises it rather than capping anything.

```
# 116_L4_RANGE_attacker
masscan -p21,22,23,53,80,443,8080 116.7.0.0/24 --rate 1000
```

masscan asks a transport-layer question and zmap's `icmp_echoscan` asks a network-layer one, so the two disagree on this range by design. Neither is lying: a host may reply on one protocol, or on one port, and not on another, so any single-probe result is a lower bound on what is out there.

traceroute: how far away a segment is, and what is between

Definition: traceroute

It sends probes with the IP time-to-live field set to 1, then 2, and so on. A router that decrements the field to zero discards the packet and returns an ICMP time-exceeded message, whose source address is the interface the probe arrived on. The result is one line per router on the path, in order, then a line for the destination itself.

```
# 116_L4_RANGE_attacker
traceroute -n 116.19.0.7
```

The **hops** figure you report for a segment is the number of routers between you and it, which is the number of lines before the destination's own line. A host on a segment one router away gives two lines, so the answer for that segment is **hops 1**. This is also the only way to find the addresses on the router-to-router links, which no sweep of a segment will ever show you: they are the middle lines of the output, and each is worth three points as a router interface.

Reading an ICMP unreachable

A router that has no route for an address replies with an ICMP network unreachable straight away. A router that has the subnet on one of its own interfaces but cannot resolve the address inside it replies with an ICMP host unreachable, after it has given up on address resolution. `ping` prints both as `Destination Host Unreachable` or `Destination Net Unreachable` rather than as a timeout, so a few probes into a candidate subnet separate three cases before you commit a full sweep to it:

```
# 116_L4_RANGE_attacker: three probes tell you which of three things a /24 is
ping -c 4 -W 2 116.31.0.77
#   replies                               -> something is there
#   Destination Net Unreachable -> no router on the path has a route for it
#   Destination Host Unreachable-> a router has the subnet and nothing is in it
```

The third case is the routed-and-empty subnet. It is not a live segment and reporting it as one costs you three points, but its .1 is a router interface and is worth three.

The Nmap Scripting Engine

Definition: Nmap Scripting Engine (NSE)

A library of scripts nmap runs against a port after it has identified what is there. `--script <name>` selects one. A script decides for itself whether it applies to a port, usually from the port number or from the service `-sV` identified, which is why a service on an unusual port needs `-sV` on the same command line before its script will run at all.

```
# 116_L4_RANGE_attacker
nmap -sV -p 2100 --script ftp-anon 116.7.0.87      # does it take an anonymous
login?
nmap -sV -p 8080 --script http-enum 116.7.0.23    # guess paths from a
fingerprint list
nmap -p 53 --script dns-zone-transfer --script-args
dns-zone-transfer.domain=example.lab 116.7.0.88
```

Talking to a service by hand

Version detection names a release for most of what this range runs and for some of it names nothing. Where nmap prints a service with no version, the version is still on the wire and the way to it is a question you ask the service yourself. That is the case for the key-value store, the proxy, the print port and the monitoring agent, and it is worth three points a time.

```
# 116_L4_RANGE_attacker
curl -sI http://116.7.0.23/           # response headers only; Server
names the software
curl -s ftp://anonymous:x@116.7.0.87:2100/pub/handover.txt # read a file off an
anonymous share
nc -w 3 116.7.0.55 22 </dev/null      # the first line a service
sends unprompted
dig @116.7.0.88 -x 116.7.0.23        # a PTR lookup: the name, and
the zone it is in
dig @116.7.0.88 example.lab AXFR     # then ask for the whole zone
telnet 116.7.0.201 23                # a login prompt, and the
device it belongs to
```

SNMP: the interface tables as a second map

Definition: SNMP, and the community string

A protocol for reading counters and configuration off network equipment over UDP. Version 2c authenticates a read with one shared string, the **community**, sent in the clear. A great deal of equipment still ships with **public** set for read access, and an agent configured that way answers anybody who asks. The values it exposes are named by OIDs, and the standard ones this range uses are **sysDescr** (what the machine is), **sysContact** (who runs it), and the **ifTable** and **ipAddressTable** families, which list its interfaces and their addresses.

An interface table is a map of the machine's own connections, so an agent on a machine with more than one interface names blocks a sweep has not reached. It also names blocks that are not routed anywhere, which is the trap in the previous section: an address in the table is an address configured on an interface, not a segment you can reach.

```
# 116_L4_RANGE_attacker
snmpget -v2c -c public 116.7.0.23 sysDescr.0 sysContact.0
snmpwalk -v2c -c public 116.7.0.23 ipAdEntAddr # every address this machine
holds
snmpwalk -v2c -c public 116.7.0.23 ifDescr     # and every interface, up or down
snmpwalk -v2c -c public 116.7.0.23 ifAdminStatus # which of them are
administratively up
```

TFTP: a file you can only fetch by name

Definition: TFTP

A file transfer protocol over UDP with no authentication, no directory listing and no command that enumerates anything. The client sends a read request naming a file and the server either sends it or answers with an error. A file on a TFTP server is therefore readable exactly when its name is already known, and learning the name is separate work somewhere else.

What sits on one is a configuration backup, because pushing the running configuration to a TFTP server nightly is what network equipment does. A switch configuration contains the credentials configured on that switch.

The file follows the organisation's own naming standard, which is its short prefix, then a site

name, then what the file is: `rvc-civic-running-config.cfg`. Knowing the standard does not give you the name, because the site and the last part are both drawn. Something else on the range names the file in full.

```
# 116_L4_RANGE_attacker: the client is interactive, so -c takes the command
cd /tmp && tftp 116.7.0.23 69 -c get rvc-civic-running-config.cfg
cat /tmp/rvc-civic-running-config.cfg
```

SMTP: asking a mail server which accounts exist

Definition: VRFY

An SMTP command that asks the server whether it knows a mailbox. RFC 5321 allows a server to answer 252, which confirms nothing, and most public servers do. A server configured to answer honestly replies 250 for an address it holds and 550 for one it does not, which turns a list of guessed names into a list of real accounts before a single password has been tried.

```
# 116_L4_RANGE_attacker: type into the session, or pipe it in as below
printf 'VRFY netadmin\r\nVRFY nosuchperson\r\nQUIT\r\n' | nc -w 5 116.7.0.23 25
# 250 ... the account exists
# 550 ... it does not
```

Redis, rsync and MQTT: three services that answer without a credential

Definition: Redis

An in-memory key-value store. It listens on 6379 and requires no authentication unless it is configured to, so a copy reachable from a network it was not meant to be on hands its whole key space to anyone who opens the socket. `KEYS *` lists every key and `GET` reads one. Its release is in the `INFO` reply under `redis_version`, which is where the version mark on that port comes from.

Definition: rsync in daemon mode

`rsync` listening on 873 serves named `modules`, each a directory tree. A module with `list = yes` and no `auth users` is readable by anybody: `rsync rsync://host/` returns the module list with its comments, and adding the module name returns the file listing. The greeting it sends is `@RSYNCD:` followed by the protocol version, which is a version of the protocol and not of the program.

Definition: MQTT

A publish-subscribe protocol for telemetry, on 1883. A broker with `allow_anonymous true` accepts any client. Subscribing to `#` subscribes to every topic there is, and a `retained` message means the broker keeps the last value published on a topic and sends it to a client that subscribes afterwards, so a subscription made now returns what was published before you arrived.

```
# 116_L4_RANGE_attacker
redis-cli -h 116.7.0.23 -p 6379 keys '*'
redis-cli -h 116.7.0.23 -p 6379 get backup:keyphrase
redis-cli -h 116.7.0.23 -p 6379 info server          # redis_version is here
rsync rsync://116.7.0.44:873/                      # the module list
rsync rsync://116.7.0.44:873/backup/                # the files in one module
rsync rsync://116.7.0.44:873/backup/restore-notes.txt /tmp/
mosquitto_sub -h 116.7.0.55 -p 1883 -t '#' -v -W 5 # every retained topic, then
stop
```

An open proxy: a connection made on your behalf

Definition: Forward proxy

A server that accepts an HTTP request naming a full URL and makes that request itself, returning the answer. A proxy that accepts requests from anybody is an open proxy, and a machine that can reach it can reach whatever the proxy can reach, with the proxy's source address on the packets rather than its own. Its answers carry a **Via** header naming it, which is how a proxy in the path is spotted from the client side.

```
# 116_L4_RANGE_attacker
curl -s -x 116.7.0.66:3128 http://116.19.0.7/      # fetched by the proxy, not by
you
printf 'GET / HTTP/1.0\r\n\r\n' | nc -w 3 116.7.0.66 3128 # its own error page
names its version
```

A print port and an agent that nmap does not know

Port 9100 is raw printing: whatever is written to the socket is printed, and a printer that speaks HP's Printer Job Language answers a few queries about itself on the same socket. **nmap** reports the port as **jetdirect** and gives no version, because the device volunteers nothing until it is asked.

Some ranges also carry a monitoring agent on a high port with a line protocol of its own. Nothing in **nmap**'s service database matches it, so version detection names the port from its own port-number list and gets the service wrong. The banner it sends on connection names the product and its release, and that is the answer.

```
# 116_L4_RANGE_attacker
printf '@PJL INFO ID\r\n' | nc -w 3 116.7.0.77 9100      # model
printf '@PJL INFO VARIABLES\r\n' | nc -w 3 116.7.0.77 9100 # FIRMWARE= is the
version
nc -w 3 116.7.0.88 9600 </dev/null                        # the agent greets
on connection
printf 'ID\r\nQUIT\r\n' | nc -w 3 116.7.0.88 9600        # and answers three
commands
```

A port that accepts a connection and then sends nothing at all, and answers nothing you send it, is one of the junk listeners a **noisy** range opens. **nmap -sV** reports it as **tcpwrapped**, and **unknown** is the service name to report for it. Deciding a port is not worth another twenty minutes is part of the exercise.

Definition: Zone transfer (AXFR)

The query type a secondary name server uses to take a full copy of a zone from the primary. A server configured to serve it to anybody hands over every record it holds in one exchange, which on this range means every segment's gateway and every machine's address at once. There is one such server on every range above **easy**.

An AXFR needs the zone's name, and the name is not something you are told: the organisation is drawn per spawn. The server holds the reverse zones as well as the forward one, which an internal name server usually does, so a PTR lookup on any address you have already found returns a name, and the name carries the zone. That is one query, and it is the query that makes the transfer possible.

Default credentials, and guessing one that is not

A telnet login prompt on this range begins with the machine's own name. That name is the organisation's short prefix, then the device family the machine stands in for, then a number, so **rvc-acc-sw3** is an access switch on the Riverton City Council estate. Each family below ships with an account already set, and the range leaves it in place on about two telnet machines in three, which makes it worth one login attempt every time you see one.

Family in the name	Device	Shipped credential
acc-sw	access switch	admin / admin
tng-rtr	branch router	cisco / cisco
wap-ac	wireless access point	root / calvin
pdu-e	rack power distribution	apc / apc
ups-net	UPS network card	upsadm / upsadm
cam-nvr	camera recorder	admin / 9999
bms-ctl	building management controller	operator / operator
kvm-ip	remote console switch	ADMIN / ADMIN

Each organisation buys from three of these eight, so a range shows you a consistent estate rather than one of everything.

Definition: Dictionary attack

Trying each entry of a list of likely passwords against one account until one is accepted. It uses no software defect: it works when the password is a word somebody has already written down, and when nothing about the service makes a failed attempt cost the attacker anything. Telnet here has no lockout, no delay after a wrong password and no record of the attempt.

Definition: hydra

A tool that drives a dictionary attack over a named protocol. **-l** is one login name, **-P** a file of passwords, **-s** the port when the service is not on its usual one, **-t** the number of connections it runs at once, and **-f** stops at the first success.

```
# 116_L4_RANGE_attacker
hydra -l netadmin -P /usr/share/minilabs/passwords.txt -t 4 -f 116.7.0.201 telnet
hydra -l admin -p admin -f 116.7.0.201 -s 2323 telnet      # one credential, on a
moved port
```


The wordlist on your machine holds fifty passwords. The shared account's is one of them, and the account's name is not something you are given. Which service names it is drawn per spawn: it may be a handover note on an anonymous FTP share, an SNMP `sysContact` field, a `VRFY` answer from the mail service, or a page the web server's index does not link to. At **normal** two of those exist on one range and at **hard** three do, so the account's name has more than one route to it and the cheapest is whichever you find first.

A way to work through a range

This is one order that works, not the only one. Adapt it to what you are handed.

1. Read your own address and gateway. At **hard** that is the whole of your starting information, and the second octet of your address is the /16 to search.
2. Find the segments. Sweep what you were given; at **hard**, sweep the /16, or probe the .1 of candidate /24s, since every segment's router interface is one. Probe a candidate a few times before committing a full sweep to it, so a routed-and-empty subnet costs you four packets rather than four minutes.
3. Traceroute to one address on each segment. That gives you the **hops** figure and the router-to-router addresses in the same command.
4. Sweep each segment twice, once with echo requests and once with a TCP probe, and compare the two lists. The addresses that appear in the second and not the first are the machines that a ping sweep misses, and on this range they are frequently the interesting ones.
5. Port scan each machine on TCP, then scan the five UDP ports this range uses. `-p-` is slower than a default scan and finds the services on ports their registry entries do not name, which about a third of this range's services are on; `-sU -p 53,69,123,161,514` finds the six services no TCP scan will ever report.
6. Run `-sV` on every open port. Record the service and the version separately; both score, and a version is worth three points, the same as the open port it sits on. Where `-sV` names a service and no version, ask that service its own version by hand.
7. Classify each machine while you still have its scan in front of you. The TTL on its reply, its MAC if it is on your segment, the name it answers to, and what it runs are four signals, and the class is what they agree on.
8. Go back to the services that let you talk to them rather than answer a probe. A web server, an FTP share, a DNS server, an SNMP agent, a mail service, a key-value store, a file-sync daemon, a broker and a proxy can each carry a finding that no port scan will produce, and which of them do is drawn with the rest of the range.
9. Take the login. The telnet prompt names a device family, and either that family's shipped credential still works or the shared account has a password in the wordlist. Whichever service on this range names that account is where the username comes from. Read what is in that account's home directory: the foothold token is there, and on a deeper range so is the key to the next machine.
10. Work out what the key opens and where its passphrase is. They are in two different places on purpose, and the second one needs no login at all: it may be the anonymous share, a

key-value store, a backup module or a retained message on a broker. When you are on a machine, run `sudo -l` before anything else; on the last one it names the command that reads the flag.

11. Run Score. It costs nothing, you can run it again, and it tells you which categories you are thin on while the range is still up.

Safety

The range is isolated. Every container is created with no Docker network, wired only to the bridges and veth links that this range creates, and there is no route from any of them to anything outside it. The scanners on the attacker are the real tools and they will do to a production network exactly what they do here, so keep every command pointed at the addresses this range gave you. Nothing here needs `sudo`: the wiring that requires privilege is done inside a short-lived helper container, so membership of the `docker` group is all the access the range asks for.

When you are finished

Run Score, then Reveal, and read what you missed against what you found. If you want another network, use Regenerate. When you are done with the range altogether, remove it with the **Teardown** action in the TUI, or by running `scripts/teardown.sh` directly.